

Deep Learning - Spring 2026

Assignment-5

From Supervised Learning to Self-Supervised Learning

A Guided Implementation of SimCLR

Item	Value
Total Marks	100
Dataset	CIFAR-10
Submission	Individual
Due Date	Checkpoint-based. Students are expected to maintain regular progress throughout the assignment. The GitHub repository commit history and checkpoint progress will be reviewed during evaluation and viva. Submissions showing only one or two large commits near the deadline may get rejected. Failure to complete checkpoints may negatively impact the evaluation of implementation understanding and originality.
Random Seed	2026

Instructor Note: This assignment is designed as a guided introduction to self-supervised learning. Students are not expected to have studied SimCLR in lectures before attempting the assignment.

Reading: “

- Read Paper: **Advancing Self-Supervised and Semi-Supervised Learning with SimCLR**, ICML 2020
 - <https://arxiv.org/abs/2002.05709>
- A very nice introduction : [Advancing Self-Supervised and Semi-Supervised Learning with SimCLR](#)
 - **NOTE:** it has the github implementation link in it. YOU are not suppose to look at it or any other code or implementation.

Checkpoint 1 — Day 3 • fixed dataset split loading • supervised baseline training • augmentation pipeline • two-view transform • augmentation visualization	Expected GitHub commits Examples: • added CIFAR-10 split loader • implemented supervised baseline • implemented augmentation pipeline • added two-view transform • generated augmentation examples Expected outputs • results/augmentation_examples.png • graphs/supervised_loss.png
Checkpoint 2 — Day 6 • encoder implementation • projection head implementation • positive and negative pair construction • similarity matrix • NT-Xent loss implementation	Expected GitHub commits Examples: • implemented projection head • implemented cosine similarity matrix • implemented NT-Xent loss • added positive pair indexing Expected outputs • results/similarity_matrix_before_training.png
Checkpoint 3 — Day 9 • SimCLR pretraining • loss curve generation • feature similarity before vs after training	Expected GitHub commits Examples: • implemented SimCLR training loop • added contrastive pretraining • generated SimCLR loss plots Expected outputs • graphs/simclr_pretraining_loss.png • results/similarity_matrix_after_training.png • Table generated in the report comparing same image's two augmented views before and after SimCLR and different images
Checkpoint 4 — Day 12 • linear probing • fine-tuning • PCA/t-SNE visualization • metrics.json • final report	Expected GitHub commits Examples: • implemented linear probing • added fine-tuning pipeline • generated PCA visualizations • completed final report Expected outputs • results/metrics.json • results/test_predictions.csv • results/random_encoder_pca_or_tsne.png • results/simclr_encoder_pca_or_tsne.png • results/finetuned_encoder_pca_or_tsne.png • Report.pdf

1. Assignment Overview

In this assignment, you will learn how a deep network can learn useful image representations without using class labels. You will begin with a normal supervised classifier trained using limited labeled data. Then you will implement a self-supervised learning method called SimCLR. Finally, you will evaluate whether the features learned by SimCLR help in image classification.

The assignment takes you step by step through the following idea:

```
Images with labels
->
  Train supervised baseline
->
  Remove labels
->
  Create two augmented views of the same image
->
  Train model to bring similar views closer
->
  Evaluate learned features using classification
```

2. Learning Objectives

1. Explain the difference between supervised and self-supervised learning.
2. Understand why two augmented views of the same image can be treated as a positive pair.
3. Implement the SimCLR training pipeline.
4. Implement the NT-Xent contrastive loss.
5. Evaluate learned representations using linear probing and fine-tuning.
6. Compare supervised learning with self-supervised pretraining.
7. Analyze learned features using loss curves, accuracy, confusion matrices, and PCA/t-SNE visualizations.

3. Dataset and Fixed Splits

You must use CIFAR-10. To make results comparable across students, the instructor/TAs will provide fixed split files. Each file contains image indices.

```
splits/
- train_ssl_unlabeled.txt
- train_labeled_10percent.txt
- val.txt
- test.txt
```

Important Rules

1. You must use the provided split files.
2. You are not allowed to create your own train/validation/test split.

3. Do not use `random_split()`.
4. During SimCLR pretraining, labels must not be used.
5. Validation data should be used for model selection.
6. Test data should be used only for final reporting.
7. Use the fixed random seed: `seed = 2026`.

4. Library Policy

You are allowed to use PyTorch and torchvision for standard components. However, SimCLR-specific components must be implemented by you.

Component	Allowed?
CIFAR-10 loading using <code>torchvision.datasets.CIFAR10</code>	Yes
<code>torch.utils.data.DataLoader</code>	Yes
Image augmentations using <code>torchvision.transforms</code>	Yes
ResNet-18 architecture from <code>torchvision</code>	Yes
Optimizers from <code>torch.optim</code>	Yes
Accuracy, confusion matrix, PCA/t-SNE using <code>sklearn</code>	Yes

Component	Must implement yourself?
Two augmented views per image	Yes
Positive and negative pair construction	Yes
Similarity matrix	Yes
NT-Xent loss	Yes
SimCLR training loop	Yes
Linear probing protocol	Yes
Fine-tuning protocol	Yes
Evaluation and analysis	Yes

Not Allowed: Do not use ready-made SimCLR/self-supervised learning libraries such as `lightly`, `solo-learn`, `VISSL`, or `pytorch-metric-learning` `NTXentLoss`. Do not copy a SimCLR implementation from GitHub, Kaggle, blogs, or other online sources or Language models.

5. Model Architecture

You must use ResNet-18 modified for CIFAR-10. You may use `torchvision.models.resnet18(weights=None)`. You may not use ImageNet pretrained weights.

```
# Allowed
model = torchvision.models.resnet18(weights=None)

# Not allowed
model = torchvision.models.resnet18(weights="IMAGENET1K_V1")
```

For CIFAR-10, modify the first convolution and remove the ImageNet-style max-pooling layer:

```
conv1: 3 x 3 convolution, stride 1, padding 1
remove maxpool
Encoder output: 512-dimensional representation
```

Module	Specification
Encoder	ResNet-18 modified for CIFAR-10; output = 512-dimensional feature vector
Projection Head	Linear(512 -> 256), ReLU, Linear(256 -> 128). Used only during SimCLR pretraining.
Classification Head	Linear(512 -> 10). Used for supervised classification, linear probing, and fine-tuning.

6. Fixed Training Settings

For the main experiment, use the following settings. If your GPU cannot support batch size 64, you may use batch size 32, but clearly mention this in your report.

Setting	Required Value
Dataset	CIFAR-10
Encoder	ResNet-18 modified for CIFAR-10
Image size	32 x 32
Batch size	64 preferred
Minimum allowed batch size	32
SimCLR pretraining epochs	50
Linear probing epochs	20
Fine-tuning epochs	20
Optimizer	Adam
Learning rate	3e-4
Temperature tau	0.5
Projection dimension	128
Random seed	2026

Include this table in your report:

Item	Value Used
Batch size	_____
SimCLR epochs	_____
Linear probing epochs	_____
Fine-tuning epochs	_____
Learning rate	_____
Optimizer	_____
Temperature	_____
GPU/CPU used	_____
Approximate training time	_____

7. Assignment Tasks

Task 0: Conceptual Warm-up - 8 Marks

Before coding, answer the following questions in your report:

1. In supervised classification, what information does the model learn from?
2. If CIFAR-10 labels are removed, do the images still contain useful visual structure? Explain.
3. If two augmented versions are created from the same original image, should their feature representations be similar or different?
4. If two images come from different original images, should their feature representations be similar or different?
5. Why might a model trained without labels still be useful for classification later?
6. What is the difference between pretraining and fine-tuning?

Task 1: Supervised Baseline with Limited Labels - 12 Marks

Train a ResNet-18 classifier from scratch using only the fixed 10% labeled training split. This baseline will show how well a normal supervised model performs when labels are limited.

```
Use:
train_labeled_10percent.txt
val.txt
test.txt
```

Requirements:

1. Train ResNet-18 from scratch.
2. Use cross-entropy loss.
3. Use only the 10% labeled split.
4. Report training loss and validation loss.
5. Report validation accuracy and final test accuracy.
6. Show the confusion matrix on the test set.

Deliverables:

```
graphs/supervised_loss.png
results/supervised_confusion_matrix.png
```

Model	Test Accuracy
Supervised ResNet-18 from scratch using 10% labels	

Task 2: Understanding Augmentations - 8 Marks

SimCLR learns by comparing two augmented views of the same image. Therefore, before training SimCLR, you must visualize the augmentations.

Use the following augmentation pipeline:

```
import torchvision.transforms as T

simclr_transform = T.Compose([
```

```

T.RandomResizedCrop(size=32, scale=(0.2, 1.0)),
T.RandomHorizontalFlip(p=0.5),
T.ColorJitter(brightness=0.4, contrast=0.4, saturation=0.4, hue=0.1),
T.RandomGrayscale(p=0.2),
T.ToTensor(),
T.Normalize(mean=(0.4914, 0.4822, 0.4465),
            std=(0.2470, 0.2435, 0.2616))
])

```

You must implement the two-view wrapper yourself:

```

class TwoViewTransform:
    def __init__(self, transform):
        self.transform = transform

    def __call__(self, x):
        view1 = self.transform(x)
        view2 = self.transform(x)
        return view1, view2

```

Required visualization: show at least 10 examples in this format:

```

Original Image | Augmented View 1 | Augmented View 2
Original Image | Augmented View 1 | Augmented View 2
...

```

Save:

```

results/augmentation_examples.png

```

Questions to answer:

1. Are the two augmented views identical?
2. Do they still represent the same object?
3. Why should SimCLR treat them as a positive pair?
4. What could go wrong if augmentations are too weak?
5. What could go wrong if augmentations are too strong?

Task 3: Feature Similarity Before Training - 8 Marks

Before training SimCLR, pass images through a random untrained ResNet-18 encoder. For a batch of images, compute cosine similarity between view 1 and view 2 of the same image, and between views from different images.

Pair Type	Average Cosine Similarity Before Training
Same image, two augmented views	
Different images	

Purpose: This step helps you understand that before self-supervised training, the model does not yet know that two views of the same image should be close in feature space.

Task 4: SimCLR Implementation - 24 Marks

Now implement SimCLR using the unlabeled training split. Labels must not be used during this task.

train_ssl_unlabeled.txt

Task 4.1: Encoder and Projection Head

Encoder: ResNet-18 modified for CIFAR-10 □ - REMEMBER ITS NOT PRE-TRAINED, its untrained.

Projection head:

Linear(512 -> 256)

ReLU

Linear(256 -> 128)

The encoder learns image features. The projection head is used only for contrastive learning.

Task 4.2: Positive and Negative Pairs

For a batch of N original images, generate two views of each image, producing 2N augmented views.

view 1 and view 2 of the same image = positive pair

all other views = negative examples

For a small batch of 4 images, include a table like this in your report:

Original Image	View 1 Index	View 2 Index	Positive Pair
image 0	0	4	yes
image 1	1	5	yes
image 2	2	6	yes
image 3	3	7	yes

Task 4.3: Similarity Matrix

Compute a cosine similarity matrix of size 2N x 2N. Visualize the similarity matrix for one small batch as a heatmap.

results/similarity_matrix_before_training.png

Answer:

1. Why is the diagonal ignored?
2. Where are the positive pairs located?
3. Why are all other entries treated as negatives?

Task 4.4: NT-Xent Contrastive Loss

Implement the NT-Xent loss yourself. For a positive pair (i, j):

$\text{loss}(i, j) = -\log [\exp(\text{sim}(z_i, z_j)) / \tau] /$

$$\sum_{k \neq i} \exp(\text{sim}(z_i, z_k) / \tau)$$

where $\text{sim}(a, b)$ = cosine similarity between a and b

$\tau = 0.5$

Do not use a library implementation of NT-Xent loss.

Task 5: SimCLR Pretraining - 12 Marks

Train SimCLR using unlabeled CIFAR-10 images. Labels must not be used.

train_ssl_unlabeled.txt

Required outputs:

graphs/simclr_pretraining_loss.png

results/similarity_matrix_after_training.png

After training, again compute feature similarities:

Pair Type	Before SimCLR	After SimCLR
Same image, two augmented views	_____	_____
Different images	_____	_____

In your report, explain whether SimCLR training increased the similarity between two augmented views of the same image.

NOTE: For this task, you must use the fixed SimCLR training settings specified in Section 6. If your GPU cannot support batch size 64, you may use batch size 32, but you must clearly mention this in your report and in metrics.json. Do not change the learning rate, temperature, augmentation pipeline, or number of epochs for the main experiment. For this task, you must use the fixed SimCLR training settings specified in Section 6. If your GPU cannot support batch size 128, you may use batch size 64, but you must clearly mention this in your report and in metrics.json. Do not change the learning rate, temperature, augmentation pipeline, or number of epochs for the main experiment.

Task 6: Linear Probe Evaluation - 10 Marks

Now evaluate whether the SimCLR encoder learned useful features. Freeze the encoder and train only a linear classifier.

Use:

train_labeled_10percent.txt

val.txt

test.txt

Experiment A: Random Encoder Linear Probe. Use a randomly initialized frozen ResNet-18 encoder and train only Linear(512 -> 10).

Experiment B: SimCLR Encoder Linear Probe. Use the SimCLR pretrained encoder, freeze the encoder, and train only Linear(512 -> 10).

Model	Encoder	Trainable Part	Test Accuracy
-------	---------	----------------	---------------

Random Linear Probe	Random frozen encoder	Linear classifier only	
SimCLR Linear Probe	SimCLR frozen encoder	Linear classifier only	

Save:

graphs/linear_probe_accuracy.png

Task 7: Fine-tuning the SimCLR Encoder - 8 Marks

Initialize the encoder using the SimCLR pretrained weights. Now train the full model end-to-end for CIFAR-10 classification using only the 10% labeled training data.

Use:

train_labeled_10percent.txt

val.txt

test.txt

Experiment	Test Accuracy
Supervised ResNet-18 from scratch using 10% labels	
Random frozen encoder + linear classifier	
SimCLR frozen encoder + linear classifier	
SimCLR pretrained encoder + full fine-tuning	

Save:

graphs/finetuning_accuracy.png

Task 8: PCA or t-SNE Feature Visualization - 5 Marks

To visualize what the encoder has learned, extract 512-dimensional features for a fixed subset of validation images.

1000 validation images

seed = 2026

Generate features from:

1. Random untrained encoder
2. SimCLR pretrained encoder
3. Fine-tuned encoder

Use PCA or t-SNE to reduce features to 2D. Labels may be used only for coloring the points in the plot. Labels must not be used during SimCLR pretraining.

Required files:

results/random_encoder_pca_or_tsne.png

results/simclr_encoder_pca_or_tsne.png

results/finetuned_encoder_pca_or_tsne.png

Questions to answer:

1. Do features from the random encoder show class-wise grouping?
2. Do features from the SimCLR encoder show better grouping?
3. Does fine-tuning improve class separation?
4. Which classes are still confused?

8. Final Required Results

Your report must include the following final table:

Model	Uses Labels During Pretraining?	Encoder Frozen?	Test Accuracy
Supervised ResNet-18, 10% labels	Yes	No	_____
Random encoder + linear probe	No	Yes	_____
SimCLR encoder + linear probe	No	Yes	_____
SimCLR encoder + fine-tuning	No during pretraining; Yes during fine-tuning	No	_____

9. Required Files

Your final submission should have the following structure:

```
FirstName_RollNumber_05/  
- rollNumber_05_task1_supervised.py  
- rollNumber_05_task2_augmentations.py  
- rollNumber_05_task3_similarity.py  
- rollNumber_05_task4_simclr.py  
- rollNumber_05_task5_linear_probe.py  
- rollNumber_05_task6_finetune.py  
- rollNumber_05_allCode.py  
  
- models/  
  - simclr_encoder.pt  
  - linear_probe.pt  
  - finetuned_model.pt  
  
- results/  
  - augmentation_examples.png  
  - similarity_matrix_before_training.png  
  - similarity_matrix_after_training.png  
  - supervised_confusion_matrix.png  
  - random_encoder_pca_or_tsne.png  
  - simclr_encoder_pca_or_tsne.png  
  - finetuned_encoder_pca_or_tsne.png  
  - metrics.json
```

- test_predictions.csv
- graphs/
 - supervised_loss.png
 - simclr_pretraining_loss.png
 - linear_probe_accuracy.png
 - finetuning_accuracy.png
- README.md
- requirements.txt
- Report.pdf

10. metrics.json Format

You must submit a machine-readable results file: results/metrics.json. Use the following format:

```
{
  "student_name": "Name",
  "roll_number": "RollNumber",
  "seed": 2026,
  "batch_size": 128,
  "simclr_epochs": 50,
  "linear_probe_epochs": 20,
  "finetuning_epochs": 20,
  "learning_rate": 0.0003,
  "temperature": 0.5,

  "supervised_10percent_test_acc": 0.0,
  "random_linear_probe_test_acc": 0.0,
  "simclr_linear_probe_test_acc": 0.0,
  "simclr_finetune_test_acc": 0.0,

  "same_view_similarity_before": 0.0,
  "different_image_similarity_before": 0.0,
  "same_view_similarity_after": 0.0,
  "different_image_similarity_after": 0.0
}
```

11. test_predictions.csv Format

Submit results/test_predictions.csv in this format:

```
image_index,true_label,predicted_label,prob_class_0,prob_class_1,...,prob_class_9
```

12. Report Requirements

Your Report.pdf should not explain every line of code. It should explain your results, observations, and analysis. The report must contain:

1. Answers to conceptual warm-up questions.
2. Supervised baseline results.
3. Augmentation examples.
4. Feature similarity before and after SimCLR.
5. SimCLR loss curve.
6. Linear probe comparison.
7. Fine-tuning comparison.
8. PCA or t-SNE visualizations.
9. Confusion matrix.
10. Final comparison table.
11. Short discussion of what SimCLR learned.
12. Any implementation issues or limitations.

13. Marking Scheme

Component	Marks
Conceptual warm-up	8
Supervised baseline	12
Augmentation visualization	8
Feature similarity analysis	8
SimCLR implementation	24
SimCLR pretraining	12
Linear probe evaluation	10
Fine-tuning	8
PCA/t-SNE visualization	5
Report and viva	5
Total	100

14. Academic Integrity

You may use PyTorch, torchvision, sklearn, matplotlib, and numpy. You may discuss concepts with classmates, but you must write your own code.

You are not allowed to copy implementations from GitHub, Kaggle, blogs, or other online sources. If you use ChatGPT or any other LLM for conceptual help, debugging help, or code suggestions, you must include a link or transcript of the conversation in your submission. You should make github repo and share it with TA, we will use it to understand how periodically the code is being updated so make regular commits

Plagiarism or copied code will result in zero marks and may lead to further disciplinary action.

GitHub Repository Requirement

Each student must create a **private GitHub repository** for this assignment and share access with the assigned TA.

The repository should be named:

```
DL_Assignment5_SimCLR_RollNumber
```

Students are expected to make **regular meaningful commits** throughout the assignment. The commit history should show gradual progress, such as:

```
initial dataset loading
Task-0
added augmentation visualization
implemented two-view transform
implemented encoder and projection head
implemented similarity matrix
implemented NT-Xent loss
added SimCLR training loop
added linear probing
added fine-tuning
added plots and report results
```

A single final commit containing the complete assignment will not be considered sufficient. The GitHub repository will be used by the instructor/TAs to understand the development process, check periodic progress, and support viva/code evaluation.

Students should not upload CIFAR-10 dataset files or large model checkpoints to GitHub. Large files should be submitted only in the final zip submission. Keep this GitHub private.

Submissions with no meaningful commit history, or repositories created only near the deadline with one or two large commits WILL BE REJECTED.

16. Instructor/TA Starter Package

The instructor/TAs should provide the following starter package:

```
assignment5_starter/
- splits/
  - train_ssl_unlabeled.txt
  - train_labeled_10percent.txt
  - val.txt
  - test.txt
- utils/
  - seed.py
  - dataset_splits.py
  - metrics.py
```

- templates/
 - metrics_template.json
 - report_template.docx
- README.md

The starter package may include code for setting random seeds, loading CIFAR-10 splits, computing accuracy, saving metrics.json, and plotting confusion matrices.

The starter package should not include the SimCLR model, NT-Xent loss, contrastive training loop, linear probing implementation, or fine-tuning implementation.

17. Expected Learning Outcome

After completing this assignment, you should understand how a model can learn meaningful visual features without labels, and how those features can later be evaluated using a downstream classification task.

Key Idea: Self-supervised learning does not mean there is no supervision. It means the supervision is created automatically from the data itself. In SimCLR, the automatically created supervision is: two augmented views of the same image should have similar representations, while views from different images should have different representations.